Yes—this is a promising research direction, but I recommend sharpening the contribution before finalizing the title. My assessment is that it could support a journal submission, including to a suitable Q3/Q4 journal, **if "risk-aware" becomes a measurable method rather than just a descriptive term.** A title alone cannot establish publishability.

I am assuming that "multi-agent" means **multiple LLM-based agents**.

1. Is your title good?

Your title, with corrected formatting, is:

Risk-Aware Multi-Agent Semantic Validation for Cross-Language Software Migration

It communicates the problem and approach clearly. However, I would clarify three things in the paper:

Term	What you should define
Risk-aware	Risk of what: semantic errors, incorrect acceptance, security regressions, or migration failure?
Multi-agent	Why are multiple agents necessary, and what do they contribute beyond a single agent with tools?
Semantic validation	Which observable behaviors are compared: outputs, exceptions, state changes, or API interactions?

My recommendation: focus on **the risk of incorrectly accepting a behavior-changing translation**, and use that risk to decide how much validation each code unit receives.

2. Does this title or idea already exist?

In the public-web searches I ran for this response on October 10, 2026, I did not find an exact match for your proposed title. This is not an exhaustive Scopus search or proof of originality.

More importantly, **closely related methods already exist**:

Related research	What it already covers	Implication for your work
TransAgent: Enhancing LLM-Based Code Translation via Fine-Grained Execution Alignment — 2026; an earlier preprint appeared in 2024	Multi-agent translation and repair, using source–target execution alignment to locate errors.	Multiple agents plus execution-based semantic repair is not sufficient novelty. (chenzhenpeng18.github.io)
AlphaTrans: A Neuro-Symbolic Compositional Approach for Repository-Level Code Translation and Validation — FSE 2025	Repository-level translation, decomposition, and multiple levels of validation.	Repository-level validation alone is already an established research direction. (research.ibm.com)

Related research	What it already covers	Implication for your work
MatchFixAgent: Language-Agnostic Autonomous Repository-Level Code Translation Validation and Repair — preprint first posted September 2025	Multi-agent semantic analysis, test generation and execution, repair, and equivalence verdicts.	**This is particularly close to your proposed topic and should be a central comparison.** (arxiv.org)
LegacyTranslate: LLM-based Multi-Agent Method for Legacy Code Translation — March 2026 preprint	API-aware PL/SQL-to-Java translation using specialized translation, grounding, and refinement agents.	API-aware legacy migration is also being studied. (arxiv.org)

My interpretation: the broad combination of *multi-agent + semantic validation + cross-language migration* overlaps substantially with existing work. Your strongest candidate differentiator is **risk-adaptive validation under a fixed computational budget**. That is a proposed research gap to investigate—not a confirmed "first-ever" contribution.

3. What advancement should you implement?

Recommended direction: risk-adaptive validation

Instead of applying the same validation process to every translated function, propose a controller that answers:

Which translated code units need stronger validation, and which validation action should be performed next?

A suitable research question would be:

Can risk-adaptive multi-agent validation reduce incorrectly accepted translations compared with uniform validation, under the same computational budget?

This is more precise and testable than simply asking whether multiple agents improve translation.

A. Define an explicit risk model

For a code unit uuu, a possible proposed formulation is:

R(u)=P^(semantic defect|xu)×I(u)R(u)=\widehat{P}(\text{semantic defect}\mid x_u)\times I(u)R(u)=P(semantic defect|xu)×I(u)

where:

- xux_uxu: features extracted from the code and available validation evidence;
- P^\widehat{P}P: estimated probability of a semantic defect;
- I(u)I(u)I(u): a predefined impact score.

Candidate features to investigate include:

- Cross-language numeric and type-conversion operations.

- Exception-handling paths.
- Mutable state and side effects.
- External API dependencies.
- Branch complexity.
- Existing test coverage.
- Disagreement among validators.

Important implementation distinction: if you only combine manually weighted features, call the result a **risk-ranking score**, not a calibrated defect probability. If you claim probability estimates, evaluate their calibration on held-out data.

B. Use risk to control validation effort

A proposed policy could be:

Estimated risk	**Validation policy**
Lower	Compilation/type checks and baseline differential tests
Medium	Additional generated inputs, boundary cases, and property checks
Higher	Targeted counterexample search, deeper execution analysis, and optional bounded formal checks
Insufficient evidence	Return “inconclusive” or request human review

Do not let “low risk” mean “no behavioral validation.”

C. Generate tests targeted at predicted risks

Rather than asking a test agent to “generate more tests,” give it a specific suspected mismatch:

“Investigate whether this translation preserves behavior for negative numeric inputs and boundary values.”

Require it to produce:

1. A concrete input.
2. Source execution results.
3. Target execution results.
4. A reproducible mismatch, if found.

Execution-aligned diagnosis and test-based validation already appear in TransAgent and MatchFixAgent. Your proposed advancement should therefore be **how risk selects and prioritizes these checks**, rather than claiming test generation itself as new. (chenzhenpeng18.github.io)

D. Make uncertainty explicit

I recommend three practical outcomes:

- **Mismatch found** — accompanied by a reproducible counterexample.
- **No mismatch found within the validation budget** — accompanied by the evidence collected.
- **Inconclusive** — insufficient evidence or unsupported behavior.

Reserve “proven equivalent” for a formally justified result within explicitly stated assumptions. For your implementation, avoid treating agent agreement as the acceptance criterion; require executable or formally checkable evidence.

4. Suggested implementation architecture

For a manageable first study, I recommend **one language direction, such as Python → Java**, with clearly bounded language features.

Source code + candidate translation

↓

Static feature extraction

↓

Risk-scoring controller

↓

Adaptive validation plan

↓

Semantic-analysis / test-generation agents

↓

Sandboxed differential execution

↓

Counterexample-guided repair

↓

Revalidation and evidence report

Possible agent roles:

1. **Semantic-analysis agent:** identifies behavior that should be preserved.
2. **Test-generation agent:** creates tests targeted at predicted risks.
3. **Repair agent:** repairs candidate translations using observed failures.

Keep risk scoring, budget accounting, test execution, and acceptance rules as explicit software components where possible. **You do not need an LLM agent for every box.**

For an initial paper, I would prioritize validating translations over building an entirely new translator.

5. What experiments would make the paper convincing?

The following is a **suggested study design**, not a journal-specific acceptance requirement.

Baselines

Compare against:

1. A single-agent validator with the same tools.
2. A fixed multi-agent validation pipeline.
3. Uniform allocation of the validation budget.
4. A simple complexity-based prioritization policy.
5. A relevant existing approach, especially MatchFixAgent or TransAgent, where artifacts and scope permit.

Use the same underlying model and comparable token, time, or monetary budgets wherever possible.

Main metrics

Metric	What to measure
Incorrect-acceptance proportion	Incorrect translations among those the system accepts
Defect-detection recall	Fraction of known faulty translations detected
Acceptance coverage	Fraction accepted rather than rejected or marked inconclusive
Validation cost	Tokens, execution time, and monetary cost
Repair success	Faulty translations repaired and passing independent evaluation
Risk calibration	Whether predicted defect probabilities match observed frequencies, if probabilities are claimed

Measure acceptance coverage alongside incorrect acceptance so that a system cannot appear successful merely by marking everything inconclusive.

Essential ablations

Remove one component at a time:

- Risk scoring.
- Adaptive budget allocation.
- Risk-targeted tests.
- Multiple agents.
- The inconclusive/abstention option.

For evaluation, I recommend:

- Real translation defects plus separately reported injected defects.

- Hidden tests not available to the repair agents.
- Project-level train/test separation.
- Repeated runs and confidence intervals.
- A small repository case study if you retain **“software migration”** in the title.

If you evaluate only isolated functions, I would use **“code translation”** rather than “software migration” to keep the title aligned with the evidence.

6. Better title options

Best overall recommendation

Risk-Adaptive Multi-Agent Semantic Validation for Cross-Language Code Translation

“Risk-adaptive” directly signals that risk changes the validation strategy.

If budget allocation is the main contribution

Budget-Aware Semantic Validation of Cross-Language Code Translation Using Risk-Guided Agents

If the study is limited to one language direction

Risk-Guided Multi-Agent Validation of Python-to-Java Code Translation

If you genuinely evaluate repository migration

Risk-Adaptive Multi-Agent Validation for Repository-Level Cross-Language Software Migration

7. Is it suitable for a Scopus Q3/Q4 journal?

My assessment: potentially yes, but I would not submit a generic translator–tester–repair-agent pipeline as the main novelty. I would build the paper around:

- An explicit risk definition.
- A reproducible adaptive validation policy.
- A comparison under equal budgets.
- Evidence that the risk mechanism adds value.
- Clearly stated limits on semantic correctness.

When choosing a journal, verify its current Scopus coverage, relevant subject category, ranking year, and whether your institution means **CiteScore quartile or SJR quartile**. These are distinct metrics, and a journal can have different quartile positions across subject categories. (elsevier.com)

Bottom line: keep the research direction, but make the contribution **“risk-guided allocation of validation effort with evidence-based acceptance”**, not merely “multiple agents for code migration.”

Which source and target programming languages are you planning to use, and will your experiments cover individual functions or complete repositories?